

Домашняя работа №5 по дисциплине «МОВС: Компьютерное зрение»

Диффузионные модели

Выполнил:

Вольхин Данил Федорович

Email:

dfvolkhin@edu.hse.ru

Дата:

08 марта 2025

😊 Первый раз в жизни делаю посадку на юпитер 😊

ANSI коды для цветов текста

```
In [1]: RED = "\033[31m"
GREEN = "\033[32m"
YELLOW = "\033[33m"
BLUE = "\033[34m"
RESET = "\033[0m" # сброс цветов до стандартных
```

Шаблоны markdown

Это комментарий в коде

```
def greet(name):
```

```
print(f"Hello, {name}!")
```

code

Установка библиотек (Необязательно)

```
In [2]: # !pip install ultralytics pycocotools numpy scipy tqdm pillow scikit-learn albumen
# !pip install -q -U denoising_diffusion_pytorch
```

Подготовка к написанию кода

Импорт библиотек

```
In [3]: print(f"{YELLOW}"+60*"-" +f"{RESET}")
print(f"Библиотеки: \n")

# Дополнительные библиотеки
import platform # Узнать версию пайтона ;)
import os
import logging
import time
import sys
import io
import random
import re
import shutil
from tqdm import tqdm
from copy import deepcopy
from functools import wraps
from typing import Any, Tuple, Union, Optional, List, Type, Callable, Dict
import collections
from collections import defaultdict
from collections import Counter
import zipfile
import rarfile
from dataclasses import dataclass
import tempfile

# Основные библиотеки
import IPython.display as ipd # Добавляет виджеты для ячеек юпитера
from IPython.display import clear_output
from IPython.display import HTML
from IPython import get_ipython
import ipykernel
print(f"python: {BLUE}{platform.python_version()}{RESET} ")
import matplotlib # Для рисунков
import matplotlib.pyplot as plt
print(f"matplotlib: {BLUE}{matplotlib.__version__}{RESET}")
import numpy as np # Для работы с массивами
print(f"numpy: {BLUE}{np.__version__}{RESET}")
import pandas as pd # Работа с таблицами
print(f"pandas: {BLUE}{pd.__version__}{RESET}")
import sklearn # Много полезного для ML
from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score
from sklearn.manifold import TSNE
print(f"sklearn: {BLUE}{sklearn.__version__}{RESET}")
import scipy
from scipy.spatial.distance import cdist, euclidean
from scipy.optimize import linear_sum_assignment
print(f"scipy: {BLUE}{scipy.__version__}{RESET}")
import pydantic # Для валидации данных
```

```

from pydantic import (BaseModel, Field, StrictStr, condecimal, StrictInt, StrictBoo
                      FilePath, DirectoryPath, ValidationError, root_validator, Con
print(f"pydantic: {BLUE}{pydantic.__version__}{RESET}")
import fastapi
from fastapi import HTTPException, status
print(f"fastapi: {BLUE}{fastapi.__version__}{RESET}")
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.optim.lr_scheduler import ReduceLROnPlateau
from torch.utils.data import DataLoader, Subset, WeightedRandomSampler, Dataset
import torch.utils.data as data
from torchvision.utils import make_grid
print(f"torch: {BLUE}{torch.__version__}{RESET}")
import torchvision
from torchvision import datasets
import torchvision.transforms as transforms
from torchvision.datasets import ImageFolder
from torchvision.transforms.functional import to_pil_image
import torchvision.models as models
from datasets import load_dataset
print(f"torchvision: {BLUE}{torchvision.__version__}{RESET}")
import requests
print(f"requests: {BLUE}{requests.__version__}{RESET}")
import PIL
from PIL import Image, ImageOps
print(f"Pillow: {BLUE}{PIL.__version__}{RESET}")
import cv2
print(f"opencv: {BLUE}{cv2.__version__}{RESET}")
import pycocotools
import pycocotools.mask as maskUtils
from pytorch_metric_learning import losses

from denoising_diffusion_pytorch import Unet
from diffusers import AutoencoderKL

from rich.theme import Theme
from rich.logging import RichHandler
from rich.console import Console
from rich.pretty import install as pretty_install
from rich.traceback import install as traceback_install

print(f"{YELLOW}"+"60*"+"-"+f"{RESET}")

```

Библиотеки:

python: 3.10.12
matplotlib: 3.10.1
numpy: 2.0.2
pandas: 2.2.3
sklearn: 1.6.1
scipy: 1.15.2
pydantic: 2.10.6
fastapi: 0.115.11
torch: 2.6.0+cu124

/home/student/projects/diffusion/.venv/lib/python3.10/site-packages/tqdm/auto.py:21:
TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html

from .autonotebook import tqdm as notebook_tqdm

torchvision: 0.21.0+cu124
requests: 2.32.3
Pillow: 11.1.0
opencv: 4.11.0

Дополнительные настройки

```
In [4]: import warnings
warnings.filterwarnings("ignore")
import logging
# Отключение логирования для cmdstanpy
logging.getLogger('cmdstanpy').setLevel(logging.ERROR)
# Полезно при разработке собственных библиотек, юпитер будет переимпортировать модуль
%load_ext autoreload
%autoreload 1
```

Кастомный setup_logging

```
In [5]: log = None

def setup_logging(clean=False, debug=False):
    global log

    if log is not None:
        return log

    try:
        if clean and os.path.isfile('setup.log'):
            os.remove('setup.log')
        time.sleep(0.1) # prevent race condition
    except:
        pass

    if sys.version_info >= (3, 9):
        logging.basicConfig(level=logging.DEBUG, format='%(asctime)s | %(levelname)s | %(message)s',
                            filename='setup.log', filemode='a', encoding='utf-8', f
```

```

else:
    logging.basicConfig(level=logging.DEBUG, format='%(asctime)s | %(levelname)s',
                        filename='setup.log', filemode='a', force=True)

console = Console(log_time=True, log_time_format='%H:%M:%S-%f', theme=Theme({
    "traceback.border": "black",
    "traceback.border.syntax_error": "black",
    "inspect.value.border": "black",
}))
pretty_install(console=console)
traceback_install(console=console, extra_lines=1, width=console.width, word_wrap=True,
                  suppress=[])
rh = RichHandler(show_time=True, omit_repeated_times=False, show_level=True, show_source=False,
                 rich_tracebacks=True, log_time_format='%H:%M:%S-%f',
                 level=logging.DEBUG if debug else logging.INFO, console=console)
rh.set_name(logging.DEBUG if debug else logging.INFO)
log = logging.getLogger("sd")
log.addHandler(rh)

return log

```

In [6]: `log = setup_logging()`

Валидация входящих данных для каждого класса

validate_with_pydantic

```

In [7]: def validate_with_pydantic(model_cls):
        """
        Декоратор для валидации данных с использованием Pydantic-модели.
        """

        def decorator(func):
            @wraps(func)
            def wrapper(*args, **kwargs):
                # Проверяем данные в аргументах функции
                try:
                    data = kwargs.get("entry", args[0] if args else None)
                    if not data:
                        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
                                            detail="No data provided for validation.")

                    # Валидация данных
                    if isinstance(data, BaseModel):
                        data = data.dict(by_alias=True)
                    validated_data = model_cls(**data)
                    # Передаем валидированные данные дальше
                    kwargs["entry"] = validated_data
                    return func(*args, **kwargs)
                except ValidationError as ve:
                    log.exception("Validation failed", exc_info=ve)
                    raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
                                        detail="Invalid data for Pydantic model.") from ve
            return wrapper

```

```
return wrapper
```

```
return decorator
```

auto_generate_docstring

```
In [8]: def auto_generate_docstring(cls: Type[BaseModel]) -> Type[BaseModel]:
        """
        Декоратор для автоматического добавления docstring в классы Pydantic.
        """

        def generate_docstring(model: Type[BaseModel]) -> str:
            """
            Генерация строки документации из описания полей модели Pydantic.
            """
            docstring = []
            for field_name, field_info in model.__fields__.items():
                field_details = f"Field '{field_name}':\n"
                if field_info.description: # Получение описания
                    field_details += f"    Description: {field_info.description}\n"
                if field_info.examples: # Получение примеров
                    field_details += f"    Examples: {field_info.examples}\n"
                docstring.append(field_details)
            return "\n".join(docstring)

        # Добавляем описание к существующему docstring
        cls.__doc__ = (cls.__doc__ or "") + "\n\n" + generate_docstring(cls)
        return cls
```

EntryGraduateModel

```
In [9]: @auto_generate_docstring
class EntryGraduateModel(BaseModel):
    """
    Класс для валидации входных данных EntryGraduateModel
    """

    model_config = ConfigDict(arbitrary_types_allowed=True)
    Prefix: Optional[StrictStr] = Field("",
                                       alias="prefix",
                                       examples=["exp1"],
                                       description="Префикс к названию модели")

    Models: list[str] = Field(...,
                             alias="models",
                             examples=[["efficientnet_b0", "efficientnet_b1"]],
                             description="Список моделей для обучения")

    NameOptimizers: list[str] = Field(...,
                                      alias="name_optimizers",
                                      examples=["Adam"],
                                      description="Лист с названиями оптимизаторов")

    SizeImg: Optional[tuple] = Field((64, 64),
                                     alias="size_img",
                                     examples=[(224, 224)],
                                     description="Размер изображения для обучения")
```

```
PathData: Optional[StrictStr] = Field("./data",
    alias="path_to_data",
    examples=["./data"],
    description="Путь к директории с данными.",
)
PathWeights: Optional[StrictStr] = Field("./weights",
    alias="path_to_weights",
    examples=["./weights"],
    description="Если сохраняем веса, то к",
)
PathMetricsTrain: Optional[StrictStr] = Field("./metrics_train",
    alias="path_to_metrics_train",
    examples=["./metrics_train"],
    description="Куда сохраняем трени",
)
PathSavePlots: Optional[StrictStr] = Field("./plot",
    alias="path_to_save_plots",
    examples=["./plot"],
    description="Куда сохраняем графики",
)
UseDevice: Optional[StrictStr] = Field(None,
    alias="use_device",
    examples=["cuda"],
    description="Какое устройство использова",
)
StartLerningRate: Optional[condecimal(gt=0.00000001, le=0.01, decimal_places=8)] = F
WeightDecay: Optional[condecimal(gt=0.00000001, le=0.01, decimal_places=8)] = F

NumTimeStemps: StrictInt = Field(20,
    alias="num_timesteps",
    examples=[20],
    description="Количество шагов диффузии")
BatchSize: Optional[StrictInt] = Field(1,
    alias="batch_size",
    examples=[10],
    description="Размер пакета при обучении")
NumWorkers: Optional[StrictInt] = Field(0,
    alias="num_workers",
    examples=[0],
    description="Кол. используемых потоков",
)
PinMemory: Optional[StrictBool] = Field(False,
    alias="pin_memory",
    examples=[False],
    description="Если True ускоряет загрузк",
)
Steps: Optional[StrictInt] = Field(10000,
    alias="steps",
    examples=[10000],
    description="Количество шагов обучения для к",
)
Seed: Optional[StrictInt] = Field(17,
    alias="seed",
    examples=[17],
    description="Сажает зерно")
```

Загрузка и предобработка данных

Загрузка и распаковка набора данных OxfordFlowers

```
In [10]: class OxfordFlowersDataset:
    def __init__(self, target_dir='oxford_flowers'):
        self.target_dir = target_dir
        os.makedirs(self.target_dir, exist_ok=True)
        self.splits = ['train', 'test', 'validation']

    def download_and_prepare(self):
        """Загрузка и организация датасета"""
        dataset = load_dataset("nelorth/oxford-flowers")

        for split in self.splits:
            if split not in dataset:
                continue

            split_dir = os.path.join(self.target_dir, split)
            os.makedirs(split_dir, exist_ok=True)

            for idx, example in enumerate(dataset[split]):
                image_bytes = example['image']
                label = str(example['label'])

                # Создаем директорию для класса
                class_dir = os.path.join(split_dir, label)
                os.makedirs(class_dir, exist_ok=True)

                # Генерируем уникальное имя файла
                file_name = f"{label}_{example.get('file_name', f'{idx}')}.jpg"
                self._save_image(image_bytes, class_dir, file_name)

    def _save_image(self, image_bytes, directory, filename):
        """Сохраняет изображение из байтов"""

        # Декодируем байты в изображение
        try:
            image = image_bytes
        except Exception as e:
            raise ValueError(f"Failed to decode image: {e}")

        # Генерация уникального имени файла
        path = os.path.join(directory, filename)

        # Сохранение изображения
        image.save(path)

    def visualize_class(self, class_id, split='train', num_images=5):
        """Визуализация изображений класса"""
```



```

class_dir = os.path.join(self.target_dir, split, str(class_id))

if not os.path.exists(class_dir):
    print(f"Class {class_id} not found in {split} split")
    return

images = sorted(os.listdir(class_dir))[:num_images]
if not images:
    print(f"No images found for class {class_id}")
    return

plt.figure(figsize=(20, 3))
for i, img_name in enumerate(images):
    img_path = os.path.join(class_dir, img_name)
    img = Image.open(img_path)

    plt.subplot(1, len(images), i+1)
    plt.imshow(img)
    plt.axis('off')
    plt.title(f"Class {class_id}\n{img_name}")

plt.tight_layout()
plt.show()

```

```

In [11]: # downloader = OxfordFlowersDataset(target_dir='data')
# downloader.download_and_prepare()
# downloader.visualize_class(class_id=42, split='train', num_images=4)

```

Формирование датасета для обучения

Класс SimpleDataset

```

In [12]: class SimpleDataset(Dataset):
def __init__(self, root_dir: str, transform: Optional[callable] = None) -> None:
    """
    Инициализация датасета для валидации и тестирования.

    :param root_dir: корневая директория с изображениями
    :param transform: преобразования для изображений
    """
    self.root_dir = root_dir
    self.transform = transform
    self.image_paths = []
    self.labels = []
    self._prepare_dataset()

def _prepare_dataset(self) -> None:
    """Сбор данных и меток из файловой структуры."""
    for root, _, files in os.walk(self.root_dir):
        for file in files:
            if file.endswith('.jpg'):
                file_path = os.path.join(root, file)
                label = int(file.split('.')[0].split('_')[0])

```

```

        self.image_paths.append(file_path)
        self.labels.append(label)

    def __len__(self) -> int:
        """Возвращает общее количество изображений."""
        return len(self.image_paths)

    def __getitem__(self, index: int) -> Tuple[torch.Tensor, torch.Tensor]:
        """Возвращает тензор изображения и соответствующую метку."""
        img_path = self.image_paths[index]
        label = self.labels[index]

        img = Image.open(img_path).convert('RGB')

        if self.transform:
            img = self.transform(img)

        return (
            img, # Тензор изображения [C, H, W]
            torch.tensor(label, dtype=torch.long) # Метка класса
        )

```

Класс ResizeWithPadding

```

In [13]: # Класс для приведения изображения к заданному размеру
class ResizeWithPadding:
    def __init__(self, size):
        self.size = size

    def __call__(self, img):
        # Приведение изображения к нужному размеру с сохранением соотношения сторон
        img.thumbnail((self.size, self.size), Image.Resampling.LANCZOS)

        # Добавление паддингов
        delta_width = self.size - img.size[0]
        delta_height = self.size - img.size[1]
        padding = (delta_width // 2, delta_height // 2, delta_width - (delta_width // 2), delta_height - (delta_height // 2))

        new_img = ImageOps.expand(img, padding, fill=(0, 0, 0))

        return new_img

```

Диффузионная модель

Класс BaseDiffusion

```

In [14]: class BaseDiffusion:
    def __init__(self, num_timesteps: int):
        self.num_timesteps = num_timesteps
        self.betas = self._get_beta_schedule(num_timesteps)
        self.alphas = 1 - self.betas

```

```

self.alphas_cumprod = torch.cumprod(self.alphas, dim=-1)

def _get_beta_schedule(self, num_diffusion_timesteps):
    """
    Создает линейное расписание для betas.

    :param num_diffusion_timesteps: Количество шагов диффузии.
    :return: Массив betas длины num_diffusion_timesteps.
    """
    assert num_diffusion_timesteps >= 20, "num_diffusion_timesteps must be at 1

    # Масштабирование для beta_start и beta_end
    scale = 1000 / num_diffusion_timesteps
    beta_start = scale * 0.0001
    beta_end = scale * 0.02

    # Создание линейного расписания betas
    betas = np.linspace(beta_start, beta_end, num_diffusion_timesteps, dtype=np

    # Проверка корректности
    assert len(betas.shape) == 1, "betas must be 1-D"
    assert (betas > 0).all() and (betas <= 1).all(), "betas must be in (0, 1]"

    # Преобразование в torch.Tensor
    betas = torch.from_numpy(betas).double()
    return betas

```

Класс ForwardDiffusion

```

In [15]: class ForwardDiffusion(BaseDiffusion):
def _extract_into_tensor(self, arr, indices, broadcast_shape):
    """
    Извлекает значения из одномерного torch tensor для батча индексов.
    :param arr: одномерный torch tensor.
    :param indices: тензор индексов, извлекаемых из arr.
    :param broadcast_shape: БОльший shape из K размерностей с размерностью батч
        равной длине indices.
    :return: тензор размерностей [batch_size, 1, ...], где shape имеет K размер
    """
    assert len(arr.shape) == 1
    res = arr.to(device=indices.device)[indices].float()
    while len(res.shape) < len(broadcast_shape):
        res = res[..., None]
    return res.expand(broadcast_shape)

def get_mean_variance(self, x0, t):
    """
    Вычисляет среднее и дисперсию распределения q(x_t | x_0).

    :param x0: Исходный тензор данных (batch_size, ...).
    :param t: Временной шаг (batch_size,).
    :return: mean (среднее), variance (дисперсия).
    """
    # Извлекаем sqrt(̄̄_t) для каждого элемента в батче

```

```

sqrt_alphas_cumprod = self._extract_into_tensor(self.alphas_cumprod.sqrt(),

# Вычисляем среднее:  $mean = \sqrt{\bar{\alpha}_t} * x_0$ 
mean = sqrt_alphas_cumprod * x0

# Извлекаем  $1 - \bar{\alpha}_t$  для каждого элемента в батче
variance = self._extract_into_tensor(1 - self.alphas_cumprod, t, x0.shape)

return mean, variance

def get_samples(self, x0, t, noise=None):
    """
    Генерирует сэмпл из распределения  $q(x_t | x_0)$ .

    :param x0: Исходный тензор данных (batch_size, ...).
    :param t: Временной шаг (batch_size,).
    :param noise: Шум (batch_size, ...). Если None, генерируется случайный шум.
    :return: Сэмпл  $x_t$ .
    """
    if noise is None:
        noise = torch.randn_like(x0) # Генерация шума, если он не передан

    # Получаем среднее и дисперсию распределения  $q(x_t | x_0)$ 
    mean, variance = self.get_mean_variance(x0, t)

    # Вычисляем сэмпл по формуле:  $x_t = mean + \sqrt{variance} * noise$ 
    samples = mean + variance.sqrt() * noise

    return samples

```

Тестирование

```

In [16]: # Тестирование Вашего кода
def test_forward_diffusion():
    fdiff = ForwardDiffusion(num_timesteps=100)
    SHAPE = [2, 20]
    x0 = torch.ones(SHAPE)
    t = torch.ones((2,)).long() * 5
    mean, variance = fdiff.get_mean_variance(x0=x0, t=t)
    assert list(mean.shape) == SHAPE
    assert list(variance.shape) == SHAPE
    assert np.allclose(mean.numpy(), np.ones(SHAPE) * 0.9820154)
    assert np.allclose(variance.numpy(), np.ones(SHAPE) * 0.03564582)

    xt = fdiff.get_samples(x0=x0, t=t)
    assert list(xt.shape) == SHAPE

    noise = torch.ones(SHAPE)
    xt = fdiff.get_samples(x0=x0, t=t, noise=noise)
    assert np.allclose(xt.numpy(), np.ones(SHAPE) * 1.1708164)
    print('Отлично, Вы справились!')

test_forward_diffusion()

```

Отлично, Вы справились!

Класс ReverseDiffusionTemplate

```
In [17]: class ReverseDiffusionTemplate(BaseDiffusion):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.alphas_cumprod_prev = torch.cat(
            [torch.tensor([1.0], device=self.betas.device), self.alphas_cumprod[:-1]
            ])

    def get_variances(self):
        variance = self.betas * (1. - self.alphas_cumprod_prev) / (1. - self.alphas_cumprod)
        variance = torch.clamp(variance, max=self.betas) # Клиппинг как в оригинале
        self.variance_clipped = variance

    def get_coeffs(self):
        sqrt_alphas = torch.sqrt(self.alphas)
        self.xt_coef = (sqrt_alphas * (1 - self.alphas_cumprod_prev)) / (1 - self.alphas_cumprod)

        sqrt_alphas_cumprod_prev = torch.sqrt(self.alphas_cumprod_prev)
        self.x0_coef = (sqrt_alphas_cumprod_prev * self.betas) / (1 - self.alphas_cumprod)
```

Класс ReverseDiffusion

```
In [18]: class ReverseDiffusion(ReverseDiffusionTemplate):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.get_variances()
        self.get_coeffs()

    def _extract_into_tensor(self, arr, indices, broadcast_shape):
        """
        Извлекает значения из одномерного torch tensor для батча индексов.
        :param arr: одномерный torch tensor.
        :param indices: тензор индексов, извлекаемых из arr.
        :param broadcast_shape: БОЛЬШОЙ shape из K размерностей с размерностью батча
                               равной длине indices.
        :return: тензор размерностей [batch_size, 1, ...], где shape имеет K размерностей
        """
        assert len(arr.shape) == 1
        res = arr.to(device=indices.device)[indices].float()
        while len(res.shape) < len(broadcast_shape):
            res = res[..., None]
        return res.expand(broadcast_shape)

    def get_x0(self, xt, eps, t):
        # Извлекаем sqrt(a_t) и sqrt(1 - a_t) из одномерных тензоров
        sqrt_alphas_cumprod = self._extract_into_tensor(self.alphas_cumprod.sqrt(),
                                                         (1 - self.alphas_cumprod).sqrt(), t, xt.shape)
        x0 = (xt - sqrt_alphas_cumprod * eps) / sqrt_alphas_cumprod
```

```

        return x0

    def get_mean_variance_reverse(self, xt, eps, t):
        x0 = self.get_x0(xt, eps, t)
        mean = (
            self._extract_into_tensor(self.xt_coef, t, xt.shape) * xt +
            self._extract_into_tensor(self.x0_coef, t, xt.shape) * x0
        )
        variance = self._extract_into_tensor(self.variance_clipped, t, xt.shape)
        return mean, variance

    def get_mean_variance(self, xt, eps, t):
        # Просто вызываем get_mean_variance_reverse
        return self.get_mean_variance_reverse(xt, eps, t)

    def get_samples(self, xt, eps, t):
        mean, variance = self.get_mean_variance_reverse(xt=xt, eps=eps, t=t)
        noise = torch.randn_like(xt, device=xt.device)
        nonzero_mask = (t != 0).float().view(-1, *[1]*(len(xt.shape)-1)))
        sample = mean + nonzero_mask * variance.sqrt() * noise
        return sample.float()

```

Тестирование

```

In [19]: # Тестирование Вашего кода
def test_reverse_diffusion():
    rdiff = ReverseDiffusion(num_timesteps=100)
    SHAPE = [2, 20]
    xt = torch.ones(SHAPE)
    eps = torch.ones(SHAPE)
    t = torch.ones((2,)).long() * 5

    x0 = rdiff.get_x0(xt=xt, eps=eps, t=t)
    assert list(x0.shape) == SHAPE
    assert np.allclose(x0.numpy(), np.ones(SHAPE) * 0.8260552)

    mean, variance = rdiff.get_mean_variance(xt=xt, eps=eps, t=t)
    assert list(mean.shape) == SHAPE
    assert list(variance.shape) == SHAPE
    assert np.allclose(mean.numpy(), np.ones(SHAPE) * 0.9467155)
    assert np.allclose(variance.numpy(), np.ones(SHAPE) * 0.007709955)

    x = rdiff.get_samples(xt, eps, t)
    assert list(x.shape) == SHAPE
    print('Отлично, Вы справились!')

test_reverse_diffusion()

```

Отлично, Вы справились!

Модель

```

In [38]: class DDPM(nn.Module):
    def __init__(self, num_timesteps: int, model: nn.Module):
        super().__init__()
        self.num_timesteps = num_timesteps
        self.forward_diffusion = ForwardDiffusion(num_timesteps=num_timesteps)
        self.reverse_diffusion = ReverseDiffusion(num_timesteps=num_timesteps)
        self.model = model
        self.shape = None

    @property
    def device(self):
        return next(self.parameters()).device

    def sample(self, num_samples, seed=None):
        """
        Генерирует сэмплы из модели.
        Если передан seed, фиксирует случайные значения для генерации шума.
        """
        assert self.shape is not None

        # Фиксируем случайные значения, если передан seed
        if seed is not None:
            torch.manual_seed(seed)

        # Генерируем начальные шумы
        x = torch.randn((num_samples, *self.shape), device=self.device, dtype=torch

        indices = list(range(self.num_timesteps))[::-1]

        for i in indices:
            t = torch.tensor([i] * num_samples, device=x.device)
            with torch.no_grad():
                eps = self.model(x, t)
                x = self.reverse_diffusion.get_samples(xt=x, eps=eps, t=t)
        return x

    def train_loss(self, x0):
        if self.shape is None:
            self.shape = list(x0.shape)[1:]
        t = torch.randint(0, self.num_timesteps, size=(x0.size(0),), device=x0.device)
        noise = torch.randn_like(x0)
        xt = self.forward_diffusion.get_samples(x0=x0, t=t, noise=noise)
        # Предсказание шума моделью
        pred_noise = self.model(xt, t) # Модель должна принимать xt и t
        # Расчет MSE между предсказанным и истинным шумом
        # print("x0 min/max:", x0.min(), x0.max()) # Должно быть в диапазоне [-1,
        # print("noise min/max:", noise.min(), noise.max()) # Должно быть нормальн
        # print("pred_noise min/max:", pred_noise.min(), pred_noise.max())
        # print("noise min/max:", noise.min(), noise.max())
        return F.mse_loss(pred_noise, noise)

    def get_latent(vae, x):
        """
        Получает латентный сэмпл из VAE.
        """

```

```
output = vae.encode(x)
z = output.latent_dist.sample()
return z
```

Обучение моделей

GraduateModel

In [129...

```
class GraduateModel:

    def __init__(self,
                  prefix: str = "",
                  name_model: str = "unet",
                  path_to_data: str = "./data",
                  path_to_weights: str = "./weights",
                  path_to_metrics_train: str = "./metrics_train",
                  name_optimizer: str = "AdamW",
                  steps: int = 10000,
                  batch_size: int = 16,
                  train_size_img: tuple = (128, 128),
                  num_timesteps: int = 20,
                  use_device: str = None,
                  start_learning_rate: float = 1e-3,
                  weight_decay: float = 1e-5,
                  pin_memory: bool = False,
                  num_workers: int = 0,
                  seed: int = 17):

        self.name_model = name_model
        self.name_model_user = f"{self.name_model}_{prefix}" if prefix != "" else self.name_model
        self.path_to_data = path_to_data
        directories_to_create = [path_to_weights, path_to_metrics_train]
        self.create_directories_if_not_exist(*directories_to_create)
        self.path_to_weights = os.path.join(path_to_weights, f"{self.name_model_user}")
        self.path_to_metrics_train = os.path.join(path_to_metrics_train, f"{self.name_model_user}")
        self.steps = steps
        self.batch_size = batch_size
        self.size_img = train_size_img
        self.num_timesteps = num_timesteps
        self.name_optimizer = name_optimizer
        self.weight_decay = float(weight_decay)
        self.model = None
        self.vae = None
        self.unet = None
        self.train_loader = None
        self.criterion = None
        self.use_device = use_device
        self.start_learning_rate = start_learning_rate
        self.pin_memory = pin_memory
        self.num_workers = num_workers
        self.seed = seed
```



```

# Получение списка доступных моделей
self.model_list = sorted(["unet"])

# Перемещение модели на GPU, если CUDA доступен
if self.use_device == None:
    self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
elif self.use_device == "cuda":
    self.device = torch.device("cuda")
elif self.use_device == "cpu":
    if self.pin_memory == True:
        raise Exception("pin_memory musn't be True with use_device cpu")
    self.device = torch.device("cpu")
else:
    raise Exception("use_device must be cuda or cpu or None")

@staticmethod
def seed_everything(seed):
    # Задаём seed для генераторов псевдослучайных чисел в Python, PyTorch и NumPy
    random.seed(seed)
    torch.manual_seed(seed)
    np.random.seed(seed)
    # Если доступна CUDA, задаём seed для GPU
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
    # Устанавливаем поведение CuDNN в детерминированный режим
    torch.backends.cudnn.deterministic = True
    # Включаем режим оптимизации CuDNN
    torch.backends.cudnn.benchmark = True
    # Создаем генератор случайных чисел PyTorch и задаём ему seed
    g = torch.Generator()
    g.manual_seed(seed)

# Функция для установки seed в DataLoader
@staticmethod
def seed_worker(worker_id):
    worker_seed = torch.initial_seed() % 2**32
    np.random.seed(worker_seed)
    random.seed(worker_seed)

def graduate(self):
    # Фиксируем зерно
    self.seed_everything(self.seed)
    # Получаем трансформер
    self.get_transform()
    # Получаем генераторы обучения, валидации и теста
    self.get_loaders()
    # Загружаем модель
    self.get_model()
    # Определяем оптимизатор, функцию потерь и планировщик
    self.get_opt_crit_sh()
    # Выводим информацию
    # print(self.__str__())
    # Обучаем
    self.train_model()

```

```

@staticmethod
def normalize(x):
    """
    Нормализует данные в диапазон [0, 1].
    """
    return (x - x.min()) / (x.max() - x.min())

def __str__(self):

    return f"""

    -----

    Выбранная модель: {self.name_model}
    Пользовательское название модели: {self.name_model_user}
    Выбранный оптимизатор: {self.name_optimizer}

    -----

    """

def create_directories_if_not_exist(self, *directories):
    for directory in directories:
        if not os.path.exists(directory):
            os.makedirs(directory)

# Функция для сохранения модели
def save_model(self):
    # Сохраняем модель
    torch.save({
        'steps': self.steps,
        'model_state_dict': self.model.state_dict(),
        'optimizer_state_dict': self.optimizer.state_dict(),
        f"{self.path_to_weights}"
    })

def save_metrics_train(self,
                       train_loss_values):
    metrics = {
        'train_loss': train_loss_values
    }
    # Сохранение метрик
    torch.save(metrics, self.path_to_metrics_train)

def get_transform(self):
    self.transform = {
        "train": transforms.Compose([
            # ResizeWithPadding(self.size_img[0]),
            transforms.Resize((self.size_img[0], self.size_img[1])),
            transforms.RandomHorizontalFlip(),
            # transforms.RandomRotation((-20, 20)),
            transforms.ToTensor(),
        ])
    }

# Функция для загрузки данных
def get_loaders(self):

```

```

# Формирование датасетов
self.train_dataset = SimpleDataset(root_dir='data/train', transform=self.tr
# Формирование загрузчиков данных
self.train_loader = DataLoader(self.train_dataset, batch_size=self.batch_si
                                num_workers=self.num_workers, pin_memory=self.pin

def get_model(self):
    # Инициализация модели
    self.vae = AutoencoderKL.from_pretrained("CompVis/stable-diffusion-v1-4", s
    self.vae.eval() # Добавляем режим оценки
    for param in self.vae.parameters():
        param.requires_grad = False # Замораживаем параметры
    if self.name_model == "unet":
        self.unet = Unet(dim=64,
                        dim_mults=(1, 2, 4, 8),
                        channels=self.vae.config.latent_channels,
                        flash_attn = True)
    self.model = DDPM(num_timesteps=self.num_timesteps, model=self.unet).to(sel
    # Загрузка весов, если путь указан
    if self.path_to_weights:
        try:
            checkpoint = torch.load(self.path_to_weights, map_location=self.dev
            self.model.load_state_dict(checkpoint['model_state_dict'])
            log.info(f"Модель успешно загружена из {self.path_to_weights}")
        except Exception as e:
            pass
    else:
        log.warning("Веса не загружены.")

    return self.model

def get_opt_crit_sh(self):
    # Определение оптимизатора
    if self.start_learning_rate is not None:
        lr = self.start_learning_rate
    else:
        lr = 0.003
    self.optimizer = optim.__dict__[f"{self.name_optimizer}"](self.model.parame

def _anneal_lr(self, step: int):
    frac_done = step / self.steps / 4
    current_lr = self.start_learning_rate * (1 - frac_done)
    for param_group in self.optimizer.param_groups:
        param_group["lr"] = current_lr
    return current_lr

def train_model(self):
    train_loss_values = []
    self.optimizer.zero_grad()
    train_iter = iter(self.train_loader)
    self.model.train()
    running_loss = 0.0
    total_samples = 0
    best_loss = float('inf')
    save_mode = False

```

```

prelast_loss = None

output_dir = "./temp"
os.makedirs(output_dir, exist_ok=True)

with tqdm(self.steps, unit="step") as pbar:
    for step in range(self.steps):

        try:
            inputs = next(train_iter)[0]
        except StopIteration:
            train_iter = iter(self.train_loader)
            inputs = next(train_iter)[0]

        inputs = inputs.to(self.device)
        # print("inputs min/max:", inputs[0].min(), inputs[0].max())
        z = get_latent(self.vae, inputs) # [B, C, H, W]
        # print("z min/max:", z.min(), z.max())
        loss = self.model.train_loss(z)
        running_loss += loss.item() * inputs.size(0)
        total_samples += inputs.size(0)
        step_loss = running_loss / total_samples

        if step == 0:
            prelast_loss = step_loss
            continue

        if step_loss > 1.5 * prelast_loss:
            log.warning("Happend save mode. Break loop step.")
            break

        train_loss_values.append(step_loss)

        loss.backward()
        self.optimizer.step()
        self.optimizer.zero_grad()

        # Сохранение лучшей модели
        if step % 1000 == 0:
            if step_loss < best_loss:
                self.save_model()
                log.info("Save model")
                best_loss = step_loss

        current_lr = self._anneal_lr(step)

        if step % 1000 == 0:
            with torch.no_grad():
                latent_samples = self.model.sample(3)
                samples = self.normalize(self.vae.decode(latent_samples).sa
                images = [np.uint8(sample.permute((1, 2, 0)).cpu().numpy().sa
                images = [Image.fromarray(image, mode='RGB') for image in i

            for idx, image in enumerate(images):
                image.save(os.path.join(output_dir, f"step_{step}_sampl

```

```

        clear_output(wait=True) # Очистка вывода

        fig, axs = plt.subplots(1, 3, figsize=(12, 4))
        for i, image in enumerate(images):
            axs[i].imshow(image)
            axs[i].set_axis_off()
        plt.tight_layout()
        plt.show()

        pbar.set_description(f"Step {step+1}/{self.steps}")
        pbar.set_postfix({'loss': running_loss/total_samples, 'lr': current_lr})

    self.save_metrics_train(train_loss_values)
    log.info("Тренировка завершена!")

```

GraduateModelPipeline

In [130...

```

@dataclass
class GraduateModelPipeline:
    entry: EntryGraduateModel

    def __post_init__(self):
        pass

    def graduate(self):

        prefix = self.entry.Prefix
        models = self.entry.Models
        name_optimizers = self.entry.NameOptimizers
        path_to_data = self.entry.PathData
        path_to_weights = self.entry.PathWeights
        use_device = self.entry.UseDevice
        start_learning_rate = self.entry.StartLearningRate
        weight_decay = self.entry.WeightDecay
        train_size_img = self.entry.SizeImg
        num_timesteps = self.entry.NumTimeStemps
        batch_size = self.entry.BatchSize
        num_workers = self.entry.NumWorkers
        pin_memory = self.entry.PinMemory
        steps = self.entry.Steps
        seed = self.entry.Seed

        path_to_metrics_train = self.entry.PathMetricsTrain
        path_to_save_plots = self.entry.PathSavePlots

        for index, name_model in enumerate(models):

            try:
                train = GraduateModel(
                    prefix=prefix,
                    name_model=name_model,
                    path_to_data=path_to_data,
                    path_to_weights=path_to_weights,
                    path_to_metrics_train=path_to_metrics_train,

```

```

        name_optimizer=name_optimizers[index],
        steps=steps,
        batch_size=batch_size,
        train_size_img=train_size_img,
        weight_decay=weight_decay,
        num_timesteps=num_timesteps,
        use_device=use_device,
        num_workers=num_workers,
        pin_memory=pin_memory,
        seed=seed)
    train.graduate()
except Exception as ex:
    log.exception("GraduateModel\n", exc_info=ex)

# Построение графиков
try:
    metrics_visualizer = MetricsVisualizer(prefix=prefix,
                                           path_to_metrics_train=path_to_me
                                           path_to_save_plots=path_to_save_
except Exception as ex:
    log.exception("MetricsVisualiser\n", exc_info=ex)

```

Визуализация графиков обучения

MetricsVisualizer

In [131...

```

class MetricsVisualizer:
    def __init__(self,
                 prefix: str = "",
                 path_to_metrics_train: str = "./metrics_train",
                 path_to_save_plots: str = None):

        self.prefix = prefix
        if path_to_save_plots:
            os.makedirs(path_to_save_plots, exist_ok=True)
        self.path_to_save_plots = path_to_save_plots if path_to_save_plots is not None else ""
        self.metrics_train_dir = os.path.join(path_to_metrics_train)
        self.train_loss_values = {}

        # Инициализируем сохранение графиков
        self.load_train_metrics()
        self.plot_metrics()

    def _load_metrics(self, directory, files_dict, key_name):
        for file in os.listdir(directory):
            if file.endswith(".pt") and f"_{self.prefix}" in file:
                metrics = torch.load(os.path.join(directory, file))
                model_name = file.replace('.pt', '').replace(f'_{self.prefix}', '')
                files_dict[model_name] = metrics[key_name]

    def load_train_metrics(self):
        self._load_metrics(self.metrics_train_dir, self.train_loss_values, 'train_loss')

```

```

def plot_metrics(self):
    fig, axs = plt.subplots(1, 1, figsize=(12, 6))

    for model, train_loss in self.train_loss_values.items():
        axs.plot(train_loss, label=model, linewidth=2)
        axs.set_title('Функция потерь на тренировке', fontsize=12)
        axs.set_xlabel('Шаг')
        axs.set_ylabel('Значение функции потерь', fontsize=12)

    # Вычисление адаптивных лимитов для оси Y
    all_values = [value for values in self.train_loss_values.values() for value in values]
    lower_limit = np.percentile(all_values, 0) # нижний предел (например, 5-й перцентиль)
    upper_limit = np.percentile(all_values, 95) # верхний предел (например, 95-й перцентиль)

    try:
        # Применение ограничений
        axs.set_ylim(lower_limit, upper_limit)
    except Exception as ex:
        pass

    # Добавление общей легенды
    handles, labels = axs.get_legend_handles_labels()
    fig.legend(handles, labels, loc='center right', fontsize=8, frameon=False)

    # Обрезаем графики, чтобы освободить место для легенды
    plt.subplots_adjust(left=0.1, right=0.8, top=0.9, bottom=0.1)
    path = os.path.join(self.path_to_save_plots, f"PlotsMetrics_{self.prefix}.png")
    plt.savefig(path, dpi=500)
    plt.show()

```

Эксперименты

Обучение

```

In [134... graduate_pipeline = validate_with_pydantic(EntryGraduateModel)(GraduateModelPipeline)

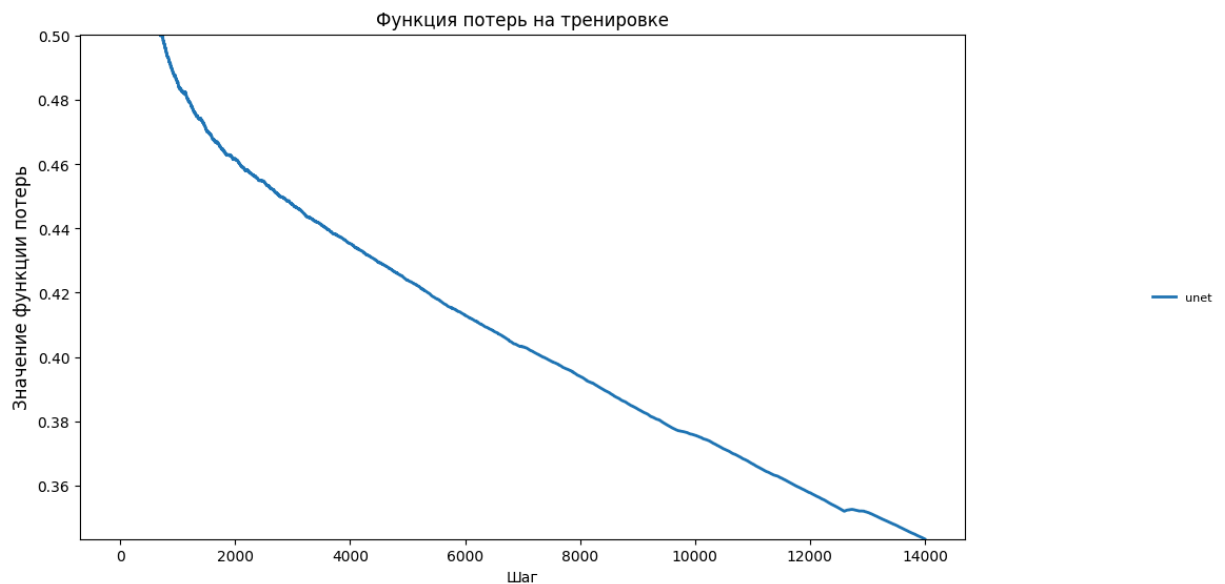
    entry = {
        "prefix": "hw5",
        "models": ["unet"],
        "name_optimizers": ["AdamW"],
        "size_img": (128, 128),
        "batch_size": 16,
        "steps": 14000,
        "num_timesteps": 1000,
        "weight_decay": 1e-5,
        "use_device": "cuda",
        "start_learning_rate": 5e-3,
        "num_workers": 0,
        "pin_memory": True,
        "seed": 17
    }

```

In [135... `graduate_pipeline.graduate()`



Step 14000/14000: : 0step [31:40, ?step/s, loss=0.343, lr=0.00075]
20:40:02-525406 INFO Тренировка завершена!



```
In [136... import imageio

def create_gifs_for_samples(image_folder, output_dir, duration=0.5):
    # Получаем список всех файлов в директории
    filenames = os.listdir(image_folder)

    # Создаем словарь для группировки изображений по сэмплам
    samples = {}

    for filename in filenames:
        if filename.endswith(".png"):
            # Разбираем имя файла, чтобы получить шаг и номер сэмпла
            parts = filename.split('_')
            step = int(parts[1])
            sample_num = int(parts[3].split('.')[0])

            # Добавляем изображение в соответствующий сэмпл
            if sample_num not in samples:
                samples[sample_num] = []
```



```

        samples[sample_num].append((step, filename))

# Создаем GIF для каждого сэмпла
gif_paths = []
for sample_num, sample_images in samples.items():
    # Сортируем изображения по шагу
    sample_images.sort(key=lambda x: x[0])

    # Создаем список изображений для GIF
    images = []
    for step, filename in sample_images:
        file_path = os.path.join(image_folder, filename)
        images.append(imageio.imread(file_path))

    # Сохраняем GIF
    gif_path = os.path.join(output_dir, f"sample_{sample_num}.gif")
    imageio.mimsave(gif_path, images, duration=duration)
    gif_paths.append(gif_path)

return gif_paths

def display_gifs(gif_paths):
    """
    Отображает GIF-анимации в одной строке с использованием HTML и CSS.
    """
    html = "<div style='display: flex;'" # Используем flexbox для горизонтального
    for gif_path in gif_paths:
        # Добавляем каждый GIF-файл в HTML
        html += f"<img src='{gif_path}' style='margin: 5px; width: 300px;'"
    html += "</div>"
    ipd.display(ipd.HTML(html))

```

```

In [137... # Создаем гифки для каждого сэмпла
image_folder = "./temp"
output_dir = "./gif"
os.makedirs(output_dir, exist_ok=True)
gif_paths = create_gifs_for_samples(image_folder, output_dir, duration=0.5)

```

```

In [138... # Отображаем гифки
if gif_paths:
    display_gifs(gif_paths)
else:
    print("Нет файлов для создания GIF-анимаций.")

```

 No description has been provided for this image

 No description has been provided for this image

 No description has been provided for this image

Инференс модели

```

In [139... vae = AutoencoderKL.from_pretrained("CompVis/stable-diffusion-v1-4", subfolder="vae")
unet = Unet(dim=64,
            dim_mults=(1, 2, 4, 8),

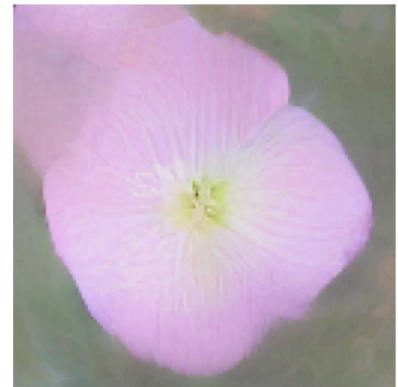
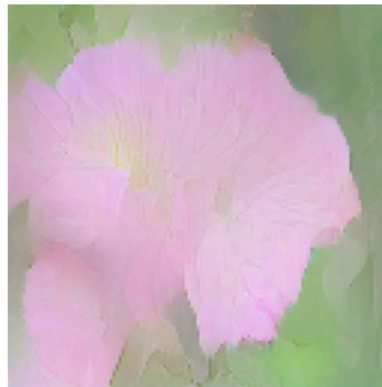
```

```

        channels=vae.config.latent_channels)
model = DDPM(num_timesteps=1000, model=unet).to("cuda")
model.shape = (4, 16, 16)
checkpoint = torch.load("./weights/unet_hw5.pt", map_location="cuda")
model.load_state_dict(checkpoint['model_state_dict'])

with torch.no_grad():
    latent_samples = model.sample(3)
    samples = vae.decode(latent_samples).sample.detach()
    samples = (samples - samples.min()) / (samples.max() - samples.min())
    images = [np.uint8(sample.permute((1, 2, 0)).cpu().numpy().clip(0, 1) * 255) for sample in samples]
    images = [Image.fromarray(image, mode='RGB') for image in images]
    fig, axs = plt.subplots(1, 3, figsize=(12, 4))
    for i, image in enumerate(images):
        axs[i].imshow(image)
        axs[i].set_axis_off()
    plt.tight_layout()
    plt.show()

```



Выводы

В целом все неплохо работает, всем советую